Negative Expectations

RSpec doesn't allow you to use the three relative by... methods or the to method with the negative expectation form, expect { ... }.not_to change { ... }. After all, when you're expecting a value *not* to change, it doesn't make sense to spell out how much it *didn't* change by or a value it *didn't* change to.

Negative expectations do, however, work with from:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
x = 5
expect { }.not_to change { x }.from(5)
```

In this example, we want x to stay at 5 before and after the block runs.

Output

Many Ruby tools write output to stdout or stderr, and RSpec includes a matcher specifically for these cases:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
expect { print 'OK' }.to output('OK').to_stdout
expect { warn 'problem' }.to output(/prob/).to_stderr
```

This matcher works by temporarily replacing the global $stdout or $stderr variable with a StringIO while it runs your expect block. This generally works well but it does have some gotchas.

For example, if you use the STDOUT constant explicitly, or spawn a subprocess that writes to one of the streams, this matcher won't work properly. Instead, you can chain to_std(out|err)_from_any_process for these situations:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb

```
expect { system('echo OK') }.to output("OK\n").to_stdout_from_any_process
```

The ...from_any_process form uses a different mechanism: it temporarily reopens the stream to write to a Tempfile. This works in more situations, but is much, much slower—30x, according to our benchmarks. Thus, you need to explicitly opt in to this slower version of the output matcher.

Your Turn

We've covered a lot of ground this chapter! From basic Ruby building blocks like strings and numbers, through deeply nested collections, to methods with side effects, you can find a matcher to suit your needs.